

Notarized Anonymity Certificates (Addendum): Extended Figures, Log Hardening Notes, and Templates

February 25, 2026

Abstract

This companion note collects material intentionally excluded from the brief “Notarized Anonymity Certificates” paper (Eval 1): extra illustrative figures for grinding patterns, additional operational notes on log equivocation hardening (witnessing, gossip, cross-logging), and short implementation-oriented templates. All content is pure L^AT_EX and remains source-only; supporting artifacts are available on request.[3]

Scope. Addendum to Eval 1.[1]

Imports. We import the core log-and-spend release discipline and manifest fields from Eval 1.[1] Transparency primitives (beacon/VRF binding, inclusion/consistency proofs) are owned by Synthesis 9.[2]

1 Extra figures and extended notes

Figure 2 shows the same effect for metric/slice/seed sweeping: the naive false-approval rate approaches 1 as the sweep library grows.

Figure 3 illustrates grinding in a simple model and how a log-enforced protocol blocks it.

Such statements can also coexist with SLSA provenance attestations when publishing software artifacts [12].

Append-only Merkle logs are vulnerable to *equivocation* (a *split-world* attack): a malicious operator can present different, internally consistent log views to different verifiers. Operational ecosystems mitigate this by (i) *gossip* that compares signed tree heads/checkpoints across clients, and/or (ii) *witnessing* in which a diverse set of third parties cosign checkpoints so that a verifier can reject un-witnessed forks.

One deployable pattern is:

1. The log periodically emits a signed checkpoint (tree head + size).
2. Witnesses fetch checkpoints, verify append-only consistency, and cosign the latest checkpoint.
3. Verifiers require a threshold of witness signatures (or verify consistency via gossip/monitors) before accepting a checkpoint as authoritative.

Witness cosigning can be implemented efficiently at scale (e.g., collective cosigning) [14], and CT-gossip variants have been studied for split-view detection [9].

Figure omitted in source-only release. Requested file: `figures/release_stop_when_pass.png`.

Figure 1: Naive stop-when-pass release gates inflate false safety approvals (unsafe releases).

Figure omitted in source-only release. Requested file:
`figures/hyperparameter_sweep_grinding.png`.

Figure 2: Metric/slice/seed sweeping inflates false approvals unless budgeted.

Cross-logging as an independent audit surface. An increasingly common hardening pattern is *cross-logging*: periodically submit the log’s signed checkpoint into an *independent* log (or log pool), so that equivocation requires compromising *multiple* operators and/or their monitoring infrastructure. This is discussed broadly in systems work on log-based transparency [11] and appears as a concrete design point in CT-adjacent proposals (e.g., pooled witnesses and log collaboration) [10]. Figure 5 is a toy model showing how even a small cross-logging rate can sharply reduce the probability a split-view persists undetected across many checkpoints.

Figure 4 is a simple simulation showing that even small witness sampling rates can make split-view attacks brittle when witnesses are diverse.

Cross-logging helps under partial witness compromise. Witnessing assumes diversity; in practice, some fraction of witnesses or monitors may be compromised or coerced. A key benefit of cross-logging is that it introduces an *independent* audit surface: even if some witnesses are silent, a fork must evade both remaining honest witnesses *and* the external log/monitoring path. Figure 6 illustrates this in a toy persistence model.

A practical witness policy template. A referee will reasonably ask: *how many witnesses, and what quorum?* While the right answer depends on your operational assumptions, a useful *sanity check* is the following conservative model: if clients accept a checkpoint only when it carries at least q cosignatures from a fixed set of W recognized witnesses, then sustaining *two inconsistent* checkpoints that both validate requires (in a worst case) at least $2q$ compromised witnesses. Thus a coarse feasibility bound is $\Pr[\text{Bin}(W, p) \geq 2q]$ when each witness is compromised independently with probability p . Figure 7 plots this bound for a few (W, q) choices. Notably, choosing $2q > W$ makes equivocation via witness compromise alone infeasible in this model, while still allowing high availability when witness downtime is modest. In practice, witness protocols and public witness directories make it straightforward to configure a diverse witness set [7].

2 Experiments (extended version)

We include three evidence types.

Spending schedules are practical. Spending is not only safe but tunable. Figure 8 compares a few standard summable schedules (a $1/t^2$ schedule and geometric schedules) against a naive stop-when-pass gate. Different schedules trade off speed (expected attempts until approval for a safe build) and conservatism, while remaining within the same total false-approval budget δ_{tot} .

Tile-backed logs make monitoring affordable. A common objection to transparency-based release discipline is that independent parties will not *actually* monitor the log at scale. Modern

Figure omitted in source-only release. Requested file: `figures/abort_after_beacon_grinding.png`.

Figure 3: Abort-after-beacon grinding can bias outcomes unless aborted attempts are logged and counted as spend.

Figure omitted in source-only release. Requested file:
`figures/split_view_witness_detection.png`.

Figure 4: Toy split-view model: detection probability rises quickly with witness diversity and modest sampling.

deployments address this with tile-backed log APIs and compact proofs, where monitors fetch fixed-size tiles (amortizing repeated requests) rather than repeatedly downloading overlapping proofs. [6, 13, 16, 8] Figure 9 is a stylized bandwidth model: as the number of audited entries grows, tile-backed auditing amortizes rapidly compared to per-entry inclusion proofs.

Inflation under retries. Figure 1 shows stop-when-pass inflation. Figure 2 shows that sweeping across many evaluation definitions inflates false approvals toward 1.

Commitment timing attacks. Figure 10 shows that post-hoc commitment (choosing the beacon round after seeing outcomes) defeats the intent of public randomness.

Grinding and partial leakage. Figure 3 and Figure 11 show that aborts and partial leakage must be treated as spend.

A Checkpoint and proof material (implementation sketch)

To keep the paper tight, we only sketch the log artifacts needed for independent verification. A practical implementation can follow tile-backed transparency-log specifications and libraries (e.g., [4, 6, 5, 7, 15]).

Checkpoint. A checkpoint is a signed statement of the log’s current tree state (log identifier, tree size, and root hash), optionally with witness cosignatures. In tile-backed ecosystems, checkpoints are designed to be *cheap to fetch* and *cheap to compare* across clients and witnesses.

Inclusion proof. For a manifest entry, the publisher supplies either: (i) a compact inclusion proof against a referenced checkpoint, or (ii) the tile path sufficient for a monitor to reconstruct the inclusion proof from cached tiles. In practice, monitors fetch the latest checkpoint and the necessary tiles; proofs are then recomputable offline.

Witnessing and cross-logging. Witness protocols allow third parties to cosign checkpoints after verifying append-only consistency, while cross-logging submits checkpoints to an independent log for additional audit surface. Public witness directories can simplify bootstrap and diversity (e.g., via public witness lists in the transparency-logs ecosystem) [7]. Both mechanisms are compatible with keeping the underlying artifacts off-chain: only hashes and manifests are logged.

Figure omitted in source-only release. Requested file: `figures/cross_logging_robustness.png`.

Figure 5: Toy robustness comparison: witnesses-only vs witnesses plus cross-logging to an independent log/monitor. Cross-logging forces an attacker to evade multiple audit surfaces, making long-lived split views brittle.

Figure omitted in source-only release. Requested file:
`figures/crosslog_witness_compromise.png`.

Figure 6: Toy split-view persistence with partial witness compromise. Dashed curves add occasional cross-logging to an independent audit surface; detection becomes markedly more likely even when a large fraction of witnesses are compromised.

B Witness policy template (minimal)

To keep artifacts compact, you can publish a *witness policy* alongside the log identifier and checkpoint origin. Below is a minimal template (informal) that is sufficient for third parties to reproduce acceptance conditions.

```
log_id: "anon-dht-eval-log"
checkpoint_origin: "anon-dht-eval/v1"
recognized_witnesses:
  - name: "w1"  pubkey: "..."
  - name: "w2"  pubkey: "..."
  - name: "w3"  pubkey: "..."
  # ... total W witnesses
quorum_q: 5
checkpoint_period: "1h"
client_rule:
  accept_checkpoint_if:
    - ">= q witness cosignatures"
    - "checkpoint in append-only log"
  restart_if:
    - "witness set changes"
    - "epoch/beacon id changes"
notes:
  - "Cross-log checkpoints weekly into an independent log"
  - "Distribute witness keys/endpoints via TUF; rotate quarterly"
```

The policy can be distributed via TUF (or your existing release channel), while the transparency log binds the *manifest hashes*. This keeps the on-net paper small while still enabling independent verification.

References

- [1] Anonymous. Notarized Anonymity Certificates Under Retries: Grinding-Resistant Evaluation via Transparency Logs, Public Randomness, and δ -Spending. Evaluation series Paper 1, 2026. (This archive.)

Figure omitted in source-only release. Requested file:
`figures/witness_policy_quorum_tradeoff.png`.

Figure 7: Conservative split-view feasibility bound under witness compromise: $\Pr[\text{Bin}(W, p) \geq 2q]$ for several witness policies (W, q) .

Figure omitted in source-only release. Requested file: `figures/spending_schedule_tradeoff.png`.

Figure 8: Speed–safety tradeoff for common summable spending schedules (stylized).

- [2] Anonymous. Transparency Primitives Interface: Beacons, VRFs, and Append-Only Logs. Series synthesis note (Synthesis 9), 2026. (This archive.)
- [3] Anonymous. Artifact policy and supporting materials for the anonymity/AnonDHT series. 2026.
- [4] Transparency Log Checkpoint Format (tlog-checkpoint). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-checkpoint>
- [5] Transparency Log Offline Proof Format (tlog-proof). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-proof>
- [6] Transparency Log Tiles API (tlog-tiles). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-tiles>
- [7] Transparency Log Witness Protocol (tlog-witness). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-witness>
- [8] Chris Hartwig and Vincent Van Doordrecht. A next-generation Certificate Transparency log built on Cloudflare Workers. Cloudflare blog, 2025. <https://blog.cloudflare.com/azul-certificate-transparency-log/>.
- [9] Rasmus Dahlberg et al. Aggregation-Based Certificate Transparency Gossip. [arXiv:1806.08817](https://arxiv.org/abs/1806.08817), 2018.
- [10] Alexandra Dirksen et al. LogPicker: Strengthening Certificate Transparency Against Covert Adversaries. *PoPETs*, 2021. <https://petsymposium.org/popets/2021/popets-2021-0066.pdf>.
- [11] Alexander Hicks. SoK: Log Based Transparency Enhancing Technologies. [arXiv:2305.01378](https://arxiv.org/abs/2305.01378), 2023.
- [12] SLSA Provenance v1.0. slsa.dev specification, 2026. Accessed 2026-02 <https://slsa.dev/spec/v1.0/provenance>
- [13] Static Certificate Transparency API. C2SP specification, 2026. Spec for tile/static CT endpoints (also mirrored at c2sp.org); accessed 2026-02 <https://c2sp.org/static-ct-api>
- [14] Ewa Syta et al. Keeping Authorities “Honest or Bust” with Decentralized Witness Cosigning. In *IEEE S&P*, 2016.
- [15] Tessera: A Go library for tile-based transparency logs. transparency-dev GitHub repository, 2026. Accessed 2026-02 <https://github.com/transparency-dev/tessera>

Figure omitted in source-only release. Requested file: `figures/tile_audit_cost.png`.

Figure 9: Stylized monitor bandwidth: tile-backed logs amortize auditing as the number of checked entries grows.

Figure omitted in source-only release. Requested file: `figures/commit_timing_attack_beacon.png`.

Figure 10: Commitment timing attacks: choosing the beacon round after partial observation enables bias.

- [16] Introducing TesseraCT. `blog.transparency.dev`, 2025. Announcement of TesseraCT (Static CT API implementation) built atop tile-backed transparency logs; accessed 2026-02 <https://blog.transparency.dev/introducing-tesseract>

Figure omitted in source-only release. Requested file: `figures/abort_partial_leakage.png`.

Figure 11: Partial leakage (seeing a noisy proxy of the outcome) can also enable grinding; log-and-spend blocks it.